

Artificial Intelligent

LAB-1

BFS

Problem 1: Graph Path Search

Find a path from 'A' to 'G' using BFS on a weighted graph (weights ignored).

Input:

```
graph = {  
  'A': ['B', 'C'],  
  'B': ['D', 'E'],  
  'C': ['F'],  
  'D': ['G'],  
  'E': ['G'],  
  'F': ['G'],  
  'G': []  
}
```

Problem 2: Maze Path Search

Find a path from 'S' to 'G' in a 2D grid using BFS. '.' = open, '#' = wall.

Input:

```
maze = [  
  ['S', '.', '.', '#', 'G'],  
  ['#', '#', '.', '#', '.'],  
  [ '.', '.', '.', '.', '.'],  
  [ '.', '#', '#', '#', '.'],  
  [ '.', '.', '.', '.', '.']  
]
```

Problem 3: Word Transformation Path

Transform 'hit' to 'cog' by changing one letter at a time, each intermediate word must be valid.

Input:

```
word_list = ["hit", "hot", "dot", "dog", "lot", "log", "cog"]
```

Problem 4: Binary Tree Level Order Traversal

Print all nodes level-wise using BFS (Level Order Traversal).

Input:

Binary Tree 1

```
    /  \
   2    3
  /\  /\
 4 5 6 7
```

DFS

Problem 1: Graph Path Search

Task:

Find a path from A to G using DFS.

Graph Representation:

```
graph = {
'A': ['B', 'C'],
'B': ['D', 'E'],
'C': ['F'],
'D': ['G'],
'E': ['G'],
'F': ['G'],
'G': []
}
```

Problem 2: Maze Solver

Task:

Find a path from 'S' to 'G' using DFS in a 2D maze.

```
maze = [
['S', '!', '!', '#', 'G'],
['#', '#', '!', '#', '!'],
['!', '!', '!', '!', '!'],
['!', '#', '#', '#', '!'],
['!', '!', '!', '!', '!']
]
```

Problem 4: Tree Traversal (Preorder)

Task:

Traverse a binary tree using DFS (Preorder).

```
      1
     /\
    2 3
   /\ /\
  4 5 6 7
```

UCS

Problem 1: Path Cost Search

Graph with Weights:

```
graph = {
  'A': [('B', 1), ('C', 4)],
  'B': [('D', 3), ('E', 2)],
  'C': [('F', 5)],
  'D': [('G', 4)],
  'E': [('G', 1)],
  'F': [('G', 2)],
  'G': []
}
```

Problem 2: Grid with Costs

```
grid = [
  [1, 1, 1, 99, 1],
  [99, 99, 1, 99, 1],
  [1, 1, 1, 1, 1],
  [1, 99, 99, 99, 1],
  [1, 1, 1, 1, 1]
]
```

Problem 3: Graph with Loops

```
graph = {
  'A': [('B', 2), ('C', 5)],
  'B': [('A', 2), ('D', 1)],
```

'C': [('A', 5), ('D', 2)],
'D': [('B', 1), ('C', 2), ('G', 3)],
'G': []
}
